

Informe Ejecutivo de Proyecto:

Exploración de los algoritmos de Clustering Profundo DEC e IDEC sobre conjunto de imágenes CIFAR-10

Universidad de Antioquia

Sebastián Castaño Quinchía

Estudiante de Astronomía

Curso Fundamentos de Deep Learning Junio 2026

1. Introducción

La segmentación no supervisada de datos complejos, como imágenes y registros de clientes, constituye un problema central en analítica de datos e inteligencia artificial, debido a la necesidad de descubrir estructuras latentes sin depender de etiquetas previas. Este tipo de tareas es especialmente relevante en contextos donde el etiquetado es costoso, ambiguo o simplemente inexistente, es decir, en problemas de aprendizaje no supervisado.

Los métodos tradicionales de agrupamiento, tales como *K-means*, los modelos de mezcla gaussiana (*GMM*), el agrupamiento jerárquico aglomerativo (*Bottom-up*) y divisivo (*Top-down*), o el agrupamiento espectral (SC), han demostrado utilidad en escenarios de baja o moderada complejidad estructural de los datos, pues suelen presentar limitaciones importantes cuando se aplican a datos de alta dimensionalidad. Debido a que estos algoritmos operan directamente sobre vectores de características construidos en el espacio de entrada, la denominada "maldición de la dimensionalidad" deteriora su capacidad para capturar y aprender relaciones complejas entre instancias y representaciones ocultas (Xie et al., 2016; Zhou et al., 2024).

Frente a estas limitaciones, surgen preguntas fundamentales en el aprendizaje automático no supervisado: cómo aprender representaciones discriminativas, cómo obtener agrupamientos más precisos a partir de ellas y cómo articular representación y *clustering* en un mismo marco de aprendizaje. Como respuesta, surgieron los algoritmos de *Deep Clustering* (DC), arquitecturas que integran técnicas de aprendizaje profundo con procesos de agrupamiento para construir representaciones latentes de menor dimensionalidad y mayor capacidad discriminativa, permitiendo que el aprendizaje de características y la formación de clústeres se retroalimenten de manera conjunta (Zhou et al., 2024).

2. Metodología

El estudio de los algoritmos de *Deep Clustering* (DC) se fundamenta en una arquitectura compuesta por un módulo de representación implementado a través de la aplicación de técnicas de aprendizaje profundo, y un módulo de agrupamiento no supervisado, que interactúan y se retroalimentan de manera

conjunta. Un esquema general de *Deep Clustering* es:

Conjunto de datos $X \rightarrow$ preprocesamiento $\rightarrow$ extractor profundo $f_\theta \rightarrow$ *embedding*
 $Z \rightarrow$ asignación de clústeres $c_i \rightarrow$ refinamiento conjunto de representación y
agrupamiento

2.1. Delimitación de implementación

Se utilizarán los algoritmos *DEC* (*Deep Embedding Clustering*) e *IDEC* (*Improved Deep Embedded Clustering*) fundacionales del *Deep Clustering* (Xie et al., 2016), para segmentación no supervisada en el conjunto de datos CIFAR-10. Describimos brevemente en qué consisten y la base de datos.

DEC aprende conjuntamente una representación latente y una asignación de clústeres mediante un autoencoder preentrenado con pérdida MSE (Error Cuadrático Medio), seguido de un refinamiento iterativo para clustering que minimiza la divergencia KL, consistente en comparar una distribución suave de asignaciones calculada mediante un kernel t-Student sobre la distancia entre cada punto y los centroides en el espacio latente, y una distribución objetivo más concentrada, construida a partir de las asignaciones de mayor confianza.

IDEC extiende el esquema de DEC incorporando la pérdida de reconstrucción del autoencoder como término de regularización durante la fase de *clustering*, por lo que se preserva la estructura global del espacio latente y evita su degradación, con función de pérdida $\mathcal{L} = \mathcal{L}_{KL} + \gamma \cdot \mathcal{L}_{rec}$.

CIFAR-10 es un *benchmark* estándar de visión por computador que contiene 60000 imágenes RGB de 32×32 píxeles distribuidas uniformemente en 10 clases: avión, automóvil, pájaro, gato, ciervo, perro, rana, caballo, barco y camión. En este trabajo se emplea la totalidad de las imágenes de forma conjunta para el proceso de agrupamiento no supervisado, prescindiendo de la partición entrenamiento/prueba dado que no se dispone de etiquetas durante el aprendizaje.

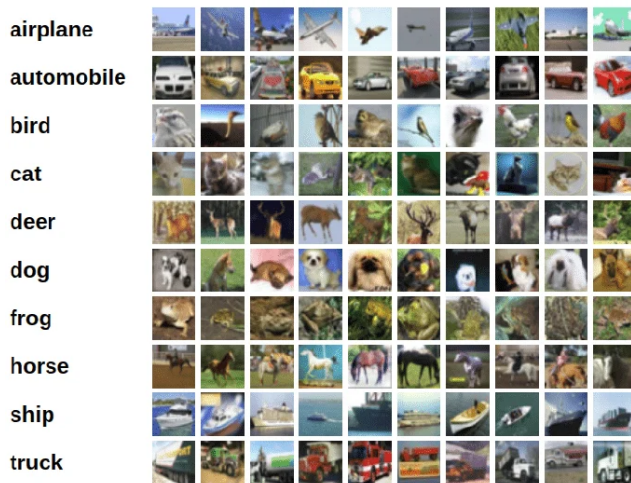


Figura 1: Enter Caption

Figura 2: Muestra representativa de las 10 clases del conjunto de datos CIFAR-10.

2.2. Validación de algoritmos

Para validar ambos algoritmos, se utilizan principalmente **métricas internas** que evalúan la calidad estructural de los clústeres sin requerir etiquetas de referencia, basándose en la cohesión intra-clúster y la separación inter-clúster en el espacio latente. Las principales son el *Silhouette Score*, que mide para cada muestra su similitud con su propio clúster frente al más cercano (un valor cercano a 1 es mejor); el *Davies-Bouldin Index*, que promedia la razón entre dispersión interna y distancia entre centroides (un valor pequeño es mejor); el *Calinski-Harabasz Index*, que relaciona la dispersión interna en los clústeres, con la dispersión intra-clúster (un valor muy grande es mejor); y el índice **CVNN** (*Clustering Validation index based on Nearest Neighbors*), propuesto por Liu et al. (2013), que evalúa la calidad del agrupamiento a partir de relaciones de vecindad en el espacio original, penalizando tanto la separación insuficiente entre clústeres como la falta de compacidad interna, sin depender de supuestos sobre la forma geométrica de los grupos.

se utilizan de forma **métricas externas** que evalúan la calidad del agrupamiento comparando las etiquetas predichas contra las etiquetas reales del conjunto de datos. Las principales son ACC (*Clustering Accuracy*), que resuelve la permutación óptima entre clústeres y clases mediante el algoritmo húngaro; NMI (*Normalized Mutual Information*), que cuantifica la información compartida entre ambas particiones normalizada por su entropía; ARI (*Adjusted Rand Index*), que mide la fracción de pares correctamente agrupados corregida por el azar; y HOM/COM (*Homogeneity/Completeness*), que evalúan respectivamente la pureza interna de cada clúster y la concentración de cada clase en un único grupo.

Adicionalmente, se construyen visualizaciones mediante t-SNE (*t-distributed Stochastic Neighbor Embedding*), que proyecta el espacio latente de alta dimensionalidad en dos dimensiones preservando las relaciones de vecindad local, permitiendo inspeccionar cualitativamente la separación entre clústeres y la cohesión interna de cada grupo.

3. Estructura del Notebook Entregado

La entrega consiste en un único notebook de Jupyter denominado `DeepC_Clustpy.ipynb`, desarrollado en Python 3 sobre Google Colab con acelerador GPU T4, que tiene tasa de transferencia o ancho de banda máximo de 320 GB/s. El notebook está organizado con los siguientes bloques:

1. **Instalación de dependencias:** Se utiliza la librería *ClustPy* [4] versión 0.0.2 estable, la cual facilita diferentes algoritmos de Deep Clustering para experimentación. De su versión en desarrollo desde [GitHub](#) importamos dos módulos que contienen la implementación de la métrica CVNN, que no está incluida en la distribución estable del paquete.

También hacemos unas importaciones globales que incluyen la librería PyTorch especializada en visión artificial y procesamiento de lenguajes naturales, y algunas métricas desde sklearn. Adicionalmente definimos una semilla para dar reproducibilidad completa mediante la fijación

de PYTHONHASHSEED, numpy, torch y las propiedades determinísticas de cuDNN; y finalmente, detecta y configura el **dispositivo de ejecución (CUDA/CPU)** a través de la variable de entorno CLUSTPY_DEVICE.

2. **Carga de datos CIFAR-10:** Se crea una función que descarga el dataset completo a través de `torchvision.datasets.CIFAR10`. En ejecuciones posteriores, los archivos se cargan desde disco sin descarga adicional.
3. **Autoencoder y preentrenamiento:** Se construye una función `crear_autoencoder()` donde se declara el autoencoder a partir de una instancia de Clustpy con los parámetros de arquitectura requeridos. También se implementa una función `pretrain_with_tracking()`, que realiza el preentrenamiento del autoencoder con pérdida MSE, registrando la pérdida media por época y soportando un scheduler `ReduceLROnPlateau` opcional que calibre la tasa de aprendizaje dinámicamente con el entrenamiento.
4. **Visualización y métricas:** Se implementan las funciones `grafico_perdida()`, genera la curva de convergencia del preentrenamiento; `metricas_externas()` que calcula doce métricas externas (ACC, PUR, HOM, COM, V-Measure, NMI, VI, FNMI, AMI, F1, RI, ARI); `metricas_internas()` que calcula las métricas internas Silhouette, Davies-Bouldin, Calinski-Harabasz y CVNN; la función `grafico_silueta_cluster()` que genera el gráfico de barras de silueta por clúster; y `grafico_TSNE_etiquetas()` que proyecta los embeddings mediante t-SNE en componentes para comparar las distribuciones y representaciones verdaderas vs predichas.
5. **Entrenamiento y validación de DEC:** Todas las funciones anteriores se generalizaron para funcionar con cualquier modelo. Después se hace un entrenamiento completo de los modelos, primero DEC y luego IDEC, consistente en cargar los datos X y etiquetas y_{true} , inicializar el autoencoder, preentrenarlo, extraer el embedding, el cálculo de métricas y gráficos de validación.
6. **Entrenamiento y validación de DEC:** Todas las funciones anteriores se generalizaron para funcionar con cualquier modelo. Después se hace un entrenamiento completo de los modelos, primero DEC y luego IDEC, consistente en cargar los datos X y etiquetas y_{true} , inicializar el autoencoder, preentrenarlo, extraer el embedding, el cálculo de métricas y gráficos de validación.

3.1. Utilidades de tiempo y carga de datos (Celdas 5–6)

La celda 5 define funciones auxiliares para sincronización CUDA y formateo de tiempos de cómputo. La celda 6 implementa la función `cargar_cifar10()`, que descarga el dataset completo (60 000 imágenes) a través de `torchvision.datasets.CIFAR10`, aplica la transformación `ToTensor()` que convierte cada imagen PIL ($H \times W \times C$, uint8) a tensor float32 ($C \times H \times W$, [0,1]) y aplana cada imagen de $3 \times 32 \times 32$ píxeles a un vector de 3072 características. La normalización z-score se aplica externamente mediante `clustpy.data.z_normalization`.

3.2. Preprocesamiento aplicado

El preprocesamiento sigue un proceso de dos etapas claramente separadas:

1. **Conversión y aplanamiento:** Durante la carga de las imágenes, con `ToTensor()` se convierte cada imagen de formato PIL ($H \times W \times C$, uint8, $[0, 255]$) a tensor PyTorch escalado ($C \times H \times W$, float32, $[0.0, 1.0]$). Posteriormente, aplanamos cada imagen $(3, 32, 32)$ en un vector de $32 * 32 * 3 = 3072$ características.
2. **Normalización z-score:** En la etapa de entrenamiento, que es donde se cargan los datos, se utiliza `z_normalization()` de ClustPy para estandarizar cada característica a media cero y desviación estándar unitaria.

4. Descripción de la Solución

Se implementan los modelos **DEC** e **IDEC** utilizando un autoencoder `FeedforwardAutoencoder` (Fully Connected) simétrico de ClustPy. La arquitectura del encoder sigue el esquema $3072 \rightarrow 1024 \rightarrow 512 \rightarrow 256 \rightarrow d$, donde d es el tamaño del embedding. El decoder reconstruye simétricamente: $d \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 3072$. La función de activación empleada es **LeakyReLU**, que evita el problema de neuronas muertas de ReLU manteniendo un gradiente negativo pequeño. No se utilizan BatchNorm ni Dropout en la configuración base. El número total de parámetros entrenables es de aproximadamente 7,6 – 7,7 millones según la dimensión de embedding elegida.

El autoencoder se entrena minimizando la pérdida MSE de reconstrucción. El objetivo es aprender un espacio latente compacto antes de aplicar el *clustering*. El clustering sobre el embedding, por defecto la inicialización de centroides se hace con K-Means y con un número de clusters de 10. En ambos algoritmos se usa un Batch Size de 256 porque son muchos datos y equilibra el gradiente con la eficiencia computacional, además, para la etapa de agrupamiento se deben doblar las épocas efectivas de preentrenamiento. En los diversos experimentos, variando los hiperparámetros y activaciones, se determinó la siguiente configuración más óptima para embedding y clustering:

- **DEC:** Se preentrena el autoencoder con un tamaño de embedding de 32 durante 80 épocas bajo un optimizador Adam con tasa de aprendizaje $lr = 10^{-3}$, en simultáneo, para agrupamiento con K-Means se usa un optimizador AdamW con tasa de aprendizaje de $lr = 5 \times 10^{-4}$ y regularización $L_2 = 10^{-6}$.
- **IDEC:** Se preentrena el autoencoder con un tamaño de embedding de 32 durante 60 épocas bajo optimizador AdamW con tasa $lr = 5 \times 10^{-4}$ con regularización $L_2 = 10^{-6}$. Para agrupamiento, se usa un optimizador Adam, con una tasa de $lr = 8 \times 10^{-5}$, además, IDEC incorpora en la pérdida de reconstrucción del autoencoder un término de regularización γ . Se controla con el hiperparámetro, el cual se define como *reconstruction_loss_weight* = 0.4.

Para ambos algoritmos se realiza la extracción del embedding, gráfico de la pérdida y validación por métricas externas e internas. También, para visualizar el espacio latente, se realiza una descomposición en

dos componentes usando t-SNE y una muestra de 10000 imágenes; y para visualizar el comportamiento de la métrica Silueta sobre cada cluster (separabilidad y compacidad) se realiza un gráfico de barras.

5. Iteraciones, Experimentos y algunos Resultados

El desarrollo del notebook siguió un proceso iterativo de ajuste progresivo. Las iteraciones más relevantes se describen a continuación:

1. **Implementación base, entrenamiento y validación:** Instalación de dependencias y carga de CIFAR-10 (60000 imágenes RGB) ~ 5 segundos con GPU T4; para DEC, preentrenamiento del autoencoder de 5 capas en 60 – 80 épocas con tiempo de ~ 2.5 minutos, y agrupamiento en ~ 3 minutos; para IDEC, preentrenamiento en ~ 2 minutos y agrupamiento en ~ 4 minutos. Finalmente, la extracción de embedding, validación por métricas externas e internas, con tiempo de ~ 2 minutos y gráfico SNE con tiempo de ~ 4.5 y las barras de la métrica Silueta con tiempo ~ 3 minutos. Este último tiempo se debe a la complejidad computacional cuadrática de la métrica de la Silueta ($O(N^2)$).

Los resultados con un embedding de tamaño 10 mostraron una ACC de aproximadamente 0.12 y una silueta débil, evidenciando que este valor era insuficiente para representar la complejidad visual de CIFAR-10, como el color y texturas.

2. **Ajuste del embedding y del optimizador en preentrenamiento:** Se aumentó el tamaño del embedding a 64 y posteriormente a 128 para DEC, sin embargo, embeddings grandes producen un mejor balance y un MSE más bajo, pero malas métricas internas, clusters menos compactos y menos separados. La regularización en preentrenamiento se varió entre $10^{-4} - 10^{-6}$, donde disminuirlo aumentaba la pérdida y no equilibraban mejor las clases. Se incluyó un scheduler para graduar la tasa de aprendizaje durante el entrenamiento, pero no tuvo ningún efecto. Con *batch_norm* activo y/o un *dropout* entre 0.05 – 0.2 se obtuvieron clases muy desbalanceadas en DEC y MSE alto, por lo que no se consideraron. Sin embargo, para IDEC, *batch_norm* activo logra balancear mucho las clases, pero no separarlos en el espacio latente, y en cambio solapa los grupos por medio de figuras (gusanos y espirales).
3. **Ajuste de la tasa de aprendizaje en agrupamiento:** Tasas de aprendizaje muy pequeñas hacen que DEC se establezca en la inicialización interna de K-Means, por lo que no refina el espacio latente. Se encontró que con un optimizador AdamW con una tasa entre $lr = 10^{-4}$ y $lr = 10^{-5}$ y una suave regularización alrededor de 10^{-6} produce clusters más equilibrados, compactos y separados. Para IDEC, la tasa más eficiente está alrededor 8×10^{-5} , y por ende la regularización es más baja, alrededor de 5×10^{-7} .
4. **Ajuste de la regularización en reconstrucción para IDEC:** En IDEC se encuentra un comportamiento similar a DEC, con la diferencia de que el modelo encuentra una mejor relación de pérdidas y buenos clusters con un embedding de 32. Respecto a la regularización, un valor entre 0.4 y 1.3 actúa como regularizador efectivo, produciendo métricas internas, clusters más

compactos y separados, y con una geometría más circular.

6. Descripción de los Resultados

6.1. Convergencia del preentrenamiento

Ambos modelos alcanzaron una convergencia estable en el preentrenamiento del autoencoder, obteniendo los siguientes resultados:

Modelo	Épocas	MSE inicial	MSE final	Tiempo	Tiempo DEC
DEC	80	0.4811	0.1610	~ 2.5 min	~ 3 min
IDEC	50	0	0	~ 2 min	~ 4 min

Notamos que la fase de *clustering* con IDEC, donde se integra tanto la función de pérdida de divergencia KL como una pérdida por reconstrucción del embedding, tiene un costo computacional mayor.

Tanto DEC como IDEC logran valores de pérdida muy similares y bajos con la configuración encontrada. Además logran un balance entre un bajo error cuadrático medio y un buen agrupamiento. Un embedding de mayor dimensión tiene mayor capacidad de reconstrucción, por lo que produce mejor balance de clases y menor MSE, pero no logra compactar y separar bien los clusters.

Ambos modelos generaron distribuciones de clústeres razonablemente balanceadas para las 10 clases de CIFAR-10 (60000 muestras en total), sin colapso completo de ningún clúster. DEC presentó clústeres en el rango de 4806 a 7215 muestras, valores que se acercan al valor real que es de 6000 muestras. IDEC mostró distribuciones en el rango de 4000 a 7500.

Métricas de evaluación comparativa

La Tabla 1 consolida las métricas externas e internas obtenidas para ambos modelos en su configuración final.

Cuadro 1: Métricas de evaluación comparativa de DEC e IDEC sobre CIFAR-10 (60 000 muestras, $K = 10$).

Métrica	Rango	DEC	IDEC
<i>Métricas externas</i>			
ACC (<i>Clustering Accuracy</i>)	$[0, 1]$	0.1857	0.2299
PUR (<i>Purity</i>)	$[0, 1]$	0.1973	0.2501
HOM (<i>Homogeneity</i>)	$[0, 1]$	0.0648	0.1045
COM (<i>Completeness</i>)	$[0, 1]$	0.0650	0.1082
VME (<i>V-Measure</i>)	$[0, 1]$	0.0649	0.1063
NMI (<i>Normalized Mutual Information</i>)	$[0, 1]$	0.0649	0.1063
FNMI (<i>Fair NMI</i>)	$[0, 1]$	0.0649	0.1063
AMI (<i>Adjusted Mutual Information</i>)	$(-1, 1]$	0.0646	0.1060
F1	$[0, 1]$	0.1303	0.1607
RI (<i>Rand Index</i>)	$[0, 1]$	0.8248	0.8193
ARI (<i>Adjusted Rand Index</i>)	$(-1, 1]$	0.0329	0.0600
VI (<i>Variation of Information</i>)	≥ 0 (min)	4.2991	4.0451
<i>Métricas internas</i>			
Silhouette Score	$[-1, 1]$	0.9517	0.7924
Davies-Bouldin Index	≥ 0 (min)	0.0627	0.5478
Calinski-Harabasz Index	$[0, \infty)$	1 949 698.88	675 293.88
CVNN	≥ 0 (min)	9.5670	14.5367

Los resultados revelan una disociación característica entre las métricas externas e internas, que es una observación frecuente y esperada en *Deep Clustering* sobre *datasets* de alta complejidad visual como CIFAR-10:

- **Métricas externas:** Ambos modelos obtienen valores muy similares y bajos. Esto es un reflejo de la dificultad que tiene los autoencoders Fully Connected que aplanan las imágenes y no capturan estructuras espaciales, colores, texturas, entre otros.
- **Métricas internas:** Revelan que los clusters tienen buena relación entre compactación y separabilidad con una métrica de silueta alta. Si se calibran bien los hiperparámetros, se logra un buen balance de clases. A pesar de obtener buenas métricas internas, y un balance de clases concebible, se pueden aún así producir clusters con solapamiento en el espacio latente, o que exhiben alguna forma (gusado, espiral). IDEC por ejemplo es muy sensible a esto y cuando se activa el hiperparámetro *batch_norm*, aunque balancea las clases, no separa en el espacio.

6.2. Archivos generados y persistencia de modelos

Al finalizar el entrenamiento de cada modelo, el notebook guarda automáticamente los artefactos listados en la Tabla 2.

Cuadro 2: Archivos generados por el notebook al finalizar el entrenamiento.

Archivo	Descripción
<code>cifar10_labels_pred.npy</code>	Etiquetas predichas por DEC (60 000,)
<code>cifar10_labels_true.npy</code>	Etiquetas verdaderas de CIFAR-10 (60 000,)
<code>cifar10_pretrain_losses.npy</code>	Pérdida MSE por época del preentrenamiento DEC
<code>cifar10_autoencoder.pt</code>	Pesos del autoencoder DEC (<i>state dict</i>)
<code>cifar10_labels_pred_IDEC.npy</code>	Etiquetas predichas por IDEC (60 000,)
<code>cifar10_labels_true_IDEC.npy</code>	Etiquetas verdaderas para IDEC (60 000,)
<code>cifar10_pretrain_losses_IDEC.npy</code>	Pérdida MSE por época del preentrenamiento IDEC
<code>cifar10_autoencoder_IDEC.pt</code>	Pesos del autoencoder IDEC (<i>state dict</i>)

Estos archivos permiten retomar la fase de validación y visualización sin necesidad de reentrenar los modelos, cargando los artefactos directamente en las celdas de validación comentadas (`cargar_embeddings()`).

7. Conclusiones y Trabajo Futuro

Este trabajo implementó y evaluó los modelos fundacionales de *Deep Clustering*, DEC e IDEC, sobre CIFAR-10, estableciendo una base sólida inicial para la exploración, análisis y comparación de estos modelos.

Las métricas externas son bajas en ambos modelos, lo que es esperable para autoencoders *feed-forward* sobre imágenes crudas de CIFAR-10. La mejora requiere arquitecturas convolucionales o métodos contrastivos que capturen invarianzas espaciales.

El *pipeline* implementado es completamente reproducible, ejecutable de principio a fin en Google Colab con GPU, y sienta las bases metodológicas para la extensión hacia algoritmos más avanzados como SCAN, CVCL o métodos de *clustering* cuántico.

Como trabajo futuro inmediato se contempla hacer una extracción de características inicial a las imágenes, como por ejemplo por medio de *PatchExtractor*, incorporar autoencoders convolucionales, explorar métodos de Deep Clustering contrastivos (SCAN, SimCLR+*clustering*) y generativos, y explorar otras métricas internas. Para reducir los tiempos en la descomposición t-SNE y mitigar las limitaciones que tiene este método, se puede pensar en otras alternativas como *UMAP Reduction*.

Referencias

- [1] Xie, J., Girshick, R., & Farhadi, A. (2016). *Unsupervised deep embedding for clustering analysis*. ICML 2016.
- [2] Guo, X., Gao, L., Liu, X., & Yin, J. (2017). *Improved deep embedded clustering with local structure preservation*. IJCAI 2017.
- [3] Zhou, X., et al. (2024). *A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions*. ACM Computing Surveys.
- [4] Leiber, C., et al. (2023). *ClustPy: A Python library for deep and non-deep clustering algorithms*. <https://github.com/collinleiber/ClustPy>
- [5] Poggiali, A., et al. (2024). *Quantum clustering methods: A comparative study*. arXiv preprint.